Лабораторная работа №4 Организация безопасного обмена данными по сети с использованием алгоритма RSA

Цель работы

Изучить принцип организации безопасного обмена данными с применением асимметричных алгоритмов шифрования на примере алгоритма RSA.

Задача работы

Используя библиотеку шифрования по алгоритму RSA, созданную на лабораторной работе №3, модифицировать код клиент-серверного приложения (лабораторные работы №1-2), реализовав поддержку обмена зашифрованными сообщениями.

Теоретическая информация

Генерация RSA-ключей

RSA-ключи генерируются следующим образом:

1. Выбираются два различных случайных простых числа p и q заданного размера (например, 1024 бита каждое).
2. Вычисляется их произведение n, которое называется **модулем**.
3. Вычисляется значение функции Эйлера $\varphi(n)$ от числа n:
4. Выбирается целое число e, взаимно простое со значением функции $\varphi(n)$. Обычно в качестве e берут простые числа, содержащие небольшое количество единичных бит в двоичной записи, например, простые числа Ферма 17, 257 или 65537.
   - Число называется **открытой экспонентой** (англ. *public exponent*)
   - Время, необходимое для шифрования с использованием быстрого возведения в степень, пропорционально числу единичных бит в e (см. алгоритм удвоения-сложения).
   - Слишком малые значения e, например 3, потенциально могут ослабить безопасность схемы RSA.
5. Вычисляется число e^{-1}, мультипликативно обратное к числу e по модулю $\varphi(n)$, то есть число, удовлетворяющее уравнению:

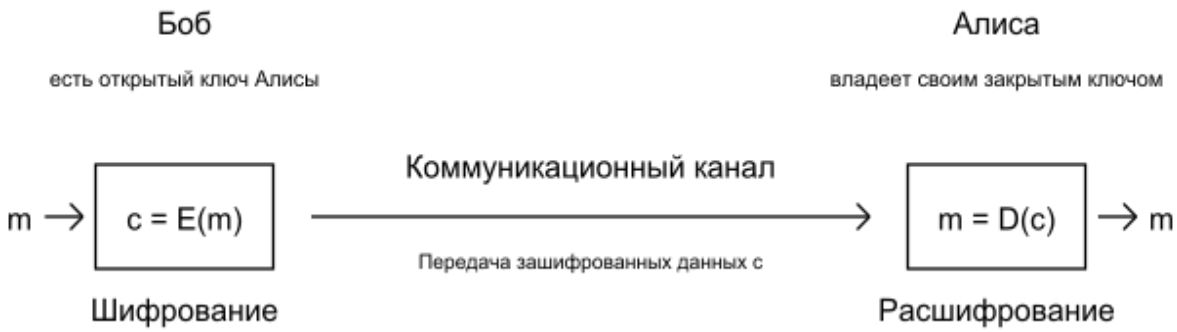
$$d = e^{-1} \, mod \, \varphi(n)$$

   - Число называется **секретной экспонентой**. Обычно, оно вычисляется при помощи расширенного алгоритма Евклида.
6. Пара $\{e, n\}$ публикуется в качестве ***открытого ключа RSA*** (англ. *RSA public key*).
7. Пара $\{d, n\}$ играет роль ***закрытого ключа RSA*** (англ. *RSA private key*) и держится в секрете.

Алгоритм передачи зашифрованного сообщения

Предположим, Боб хочет послать Алисе сообщение m.

Сообщениями являются целые числа в интервале от 0 до $n-1$, т.е $m \in \mathbb{Z}_n$.



Алгоритм:

- Взять открытый ключ $\{e,\ n\}$ Алисы
- Взять открытый текст m
- Зашифровать сообщение с использованием открытого ключа Алисы:

$$c = E(m) = m^e mod\ n$$

Алгоритм:

- Принять зашифрованное сообщение c
- Взять свой закрытый ключ $\{d,\ n\}$
- Применить закрытый ключ для расшифровывания сообщения:

$$m = D(c) = c^d mod\ n$$

Пример

Этап	Описание операции	Результат операции
Генерация ключей	Выбрать два простых различных числа	$p = 3557$ $q = 2579$
	Вычислить модуль (произведение)	$n = p \cdot q = 3557 \cdot 2579 = 9173503$
	Вычислить функцию Эйлера	$\varphi(n) = (p-1)(q-1) = 9167368$
	Выбрать открытую экспоненту	$e = 3$
	Вычислить секретную экспоненту	$d = e^{-1} \mod \varphi(n)$ $d = 6111579$
	Опубликовать *открытый ключ*	$\{e, n\} = \{3, 9173503\}$
	Сохранить *закрытый ключ*	$\{d, n\} = \{6111579, 9173503\}$
Шифрование	Выбрать текст для зашифровки	$m = 111111$
	Вычислить шифротекст	$c = E(m)$ $= m^e \mod n$ $= 111111^3 \mod 9173503$ $= 4051753$
Расшифрование	Вычислить исходное сообщение	$m = D(c) =$ $= c^d \mod n$ $= 4051753^{6111579} \mod 9173503$ $= 111111$

Задание

Результатом выполнения лабораторной работы должен стать модифицированный код клиент-серверного приложения, шифрующий передаваемые данные по алгоритму RSA. Приложение должно обладать следующей функциональностью:

1. При запуске сервер и клиент генерируют свою пару из открытого и закрытого ключей.
2. После соединения по протоколу TCP клиент первым сообщением передает серверу свой открытый ключ.
3. Сервер отвечает на это сообщение, передавая клиенту открытый ключ сервера.
4. Клиент считывает пользовательский ввод из консоли, шифрует его используя открытый ключ сервера и отправляет зашифрованное сообщение по сети.
5. Сервер принимает сообщение, расшифровывает и отправляет клиенту ответ, зашифрованный открытым ключом клиента.
6. Клиент принимает ответ сервера, расшифровывает его, выводит на экран и отключается от сервера.

Допускается использование для шифрования и расшифровывания данных криптосервисов .NET Framework, в частности класса RSACryptoServiceProvide. Описание класса представлено по ссылке: https://msdn.microsoft.com/ru-ru/library/system.security.cryptography.rsacryptoserviceprovider(v=vs.110).aspx

Список литературы

1. Мао, В. Современная криптография: теория и практика : Пер. с англ. / В. Мао. – М. : Издательский дом "Вильямс", 2005. – 768 с.
2. Шнайер, Б. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си / Б. Шнайер. – М. : Триумф, 2002. – 816 с.
3. PKCS #1 v2.1: RSA Cryptography Standard. – Bedford : RSA Laboratories, 2002. – 61 p.